

Contents

KẾ HOẠCH CHI TIẾT — Monthly Report + MCP Connections 1

1. Tóm tắt điều hành 1

2. Hiện trạng kiến trúc (As-is) 1

3. Mục tiêu & phạm vi 3

4. Kiến trúc To-be 3

5. Thiết kế chi tiết 3

6. Tổng hợp thay đổi theo file 7

7. Bảo mật 7

8. Triển khai / DevOps 8

9. Phân kỳ & cột mốc 8

10. Câu hỏi mở cần Duy chốt (M0) 8

11. Kế hoạch kiểm thử (verify) 8

KẾ HOẠCH CHI TIẾT — Monthly Report + MCP Connections

Phạm vi: Didi AI Tool (Next.js control plane) ↔ queo-solution-agent (FastAPI agent “Quéo”) **Ngày:** 2026-06-15 · **Tác giả phân tích:** dựa trên đọc trực tiếp source của cả 2 repo **Trạng thái:** Draft v1 — chờ Duy review trước khi code

1. Tóm tắt điều hành

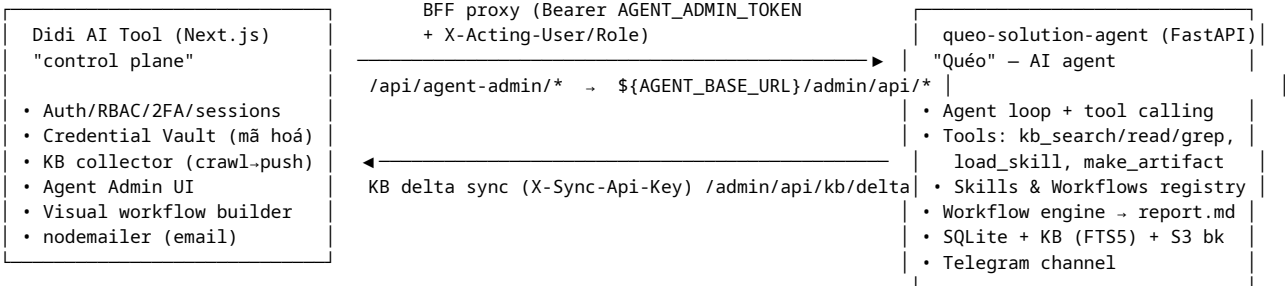
Mục tiêu: cho phép user vào Didi bấm **“Tạo report tháng”**, agent Quéo lấy số liệu (Tableau) + thông tin (folder Wealth Solution / KB) rồi xuất report **PDF + Markdown**. Nội dung report **không hardcoded** — được điều khiển bởi **skill/workflow do user upload**.

Phát hiện quan trọng nhất sau khi đọc code: ~70-80% hạ tầng đã có sẵn trong agent. Cụ thể, agent đã có: - Cơ chế **upload skill/workflow** (source='admin', cột content_override, endpoint POST /admin/api/skills & POST /admin/api/workflows). - **Workflow engine** chạy từng step qua agent loop rồi tự tổng hợp ra artifact report.md (app/core/workflows/engine.py). - **Trigger run + poll + tải artifact:** POST /admin/api/workflows/{id}/run, GET /admin/api/runs/{id}, GET /admin/api/runs/{id}/artifacts/{filename}. - **BFF + RBAC + audit** giữa Didi và agent đã hoàn chỉnh.

=> **Gap thực sự cần xây = 3 mảng:** | Mảng | Bản chất | Khối lượng | |—|—|—| | **A. MCP Connections** (Tableau, Atlas) | Agent chưa có MCP client → không lấy được số liệu live. **Đây là phần net-new lớn nhất.** | Lớn | | **B. Upload skill (UX)** | Data layer đã hỗ trợ; thiếu UX upload file .md/.zip + render menu. | Nhỏ-Vừa | | **C. Monthly Report** | = 1 workflow upload được + nút trigger trong Didi + render **PDF**. | Vừa |

2. Hiện trạng kiến trúc (As-is)

2.1 Hai hệ thống



- **Didi** chạy local/server mode (AUTH_MODE), role: viewer < operator < superadmin. Mọi call ra agent đi qua route BFF src/app/api/agent-admin/[...path]/route.ts, RBAC định nghĩa ở src/lib/rbac/agent-admin.ts.
- **Agent** headless, cấu hình qua env + config/persona.md + SQLite + /admin/api/**. LLM = GreenNode MaaS (OpenAI-compatible).

2.2 Cơ chế tool của agent (điểm tích hợp MCP)

app/core/tools/registry.py → ToolRegistry._build() trả về dict {name: ToolSpec(name, description, parameters, handler)} **tĩnh**. Agent loop dùng: - tools.schemas() → nạp vào function-calling của LLM (agent_loop.py:162) - tools.execute(name, args, ctx) → chạy handler (agent_loop.py:190, 254)

=> **Để thêm tool MCP, chỉ cần “tiêm” thêm ToolSpec vào self._tools** — agent loop tự động thấy và gọi được, **không phải sửa agent loop**.

Tool make_artifact(filename, content) đã ghi file vào \${STATE_DIR}/artifacts/{run_id}/ và append vào ctx.artifacts (md/text, ≤2MB, ≤20 file/run).

2.3 Skill & Workflow registry (điểm tích hợp Upload)

app/db/migrations/0001_init.sql:

```
CREATE TABLE skills (
  skill_id TEXT PRIMARY KEY, name, description, triggers,
  source TEXT CHECK(source IN ('kb','admin')), -- 'admin' = upload/tạo qua UI
  kb_path TEXT, content_override TEXT, -- content_override = nội dung lưu thẳng DB
  enabled, command_alias, show_in_menu, updated_at, updated_by );
CREATE TABLE workflows ( workflow_id PRIMARY KEY, ..., source CHECK(source IN ('kb','admin')),
  kb_path, content_override, schedule, enabled, command_alias, show_in_menu, ... );
CREATE TABLE workflow_runs ( id, workflow_id, trigger CHECK(IN 'telegram','admin','schedule','api'),
  triggered_by, params, status CHECK(IN 'queued','running','succeeded','failed','cancelled'),
  log, artifacts, started_at, finished_at, created_at );
```

SkillRegistry.load() / WorkflowRegistry.load() **ưu tiên content_override** rồi mới đến file kb_path. → Upload = ghi content_override, source='admin'.

2.4 Workflow engine = bộ máy chạy report (đã có)

WorkflowEngine.start() → tạo workflow_runs → chạy spec.steps (parse từ heading ## Quy trình/Steps, danh sách đánh số) lần lượt qua agent_loop.run(ctx, mode="workflow_step") → gom outputs → _write_report() gọi LLM deep tổng hợp thành artifacts/{run_id}/report.md → (Telegram) send_document. Giới hạn: WORKFLOW_MAX_STEPS=15, WORKFLOW_TOTAL_TIMEOUT_SECONDS=1800, WORKFLOW_MAX_CONCURRENT=2.

2.5 Bảng “Đã có (tái dùng)” vs “Phải xây”

Năng lực	Đã có?	File/endpoint
Upload skill/workflow (lưu DB)	[có] Data + API	POST /admin/api/skills, POST /admin/api/workflows (main.py:582,667)
Trigger chạy report	[có]	POST /admin/api/workflows/{id}/run (main.py:1082)
Poll trạng thái run	[có]	GET /admin/api/runs/{id} (main.py:1056)
Tải artifact report	[có]	GET /admin/api/runs/{id}/artifacts/{filename} (main.py:1073)
Sinh report.md	[có]	engine.py:_write_report
Lịch chạy định kỳ (cron)	[có]	Cột workflows.schedule + app/services/cron_scheduler.py

Năng lực	Đã có?	File/endpoint
BFF + RBAC + audit	[có]	Didi agent-admin/[...path]/route.ts + rbac/agent-admin.ts
Gửi email artifact	[có] (Didi)	nodemailer đã có trong package.json
MCP client/host trong agent	[thiếu]	phải xây
Lưu cấu hình + secret MCP	[thiếu]	phải xây (bảng + mã hoá)
Admin API /admin/api/mcp/*	[thiếu]	phải xây
UI Connections (MCP) trong Didi	[thiếu]	phải xây
UX upload file skill (.md/.zip)	[một phần] một phần (đã POST content_override)	bổ sung file-picker + parse
Render PDF	[thiếu]	phải xây (engine chỉ ra .md)
Trang Reports + nút “Tạo report tháng”	[thiếu]	phải xây (Didi)

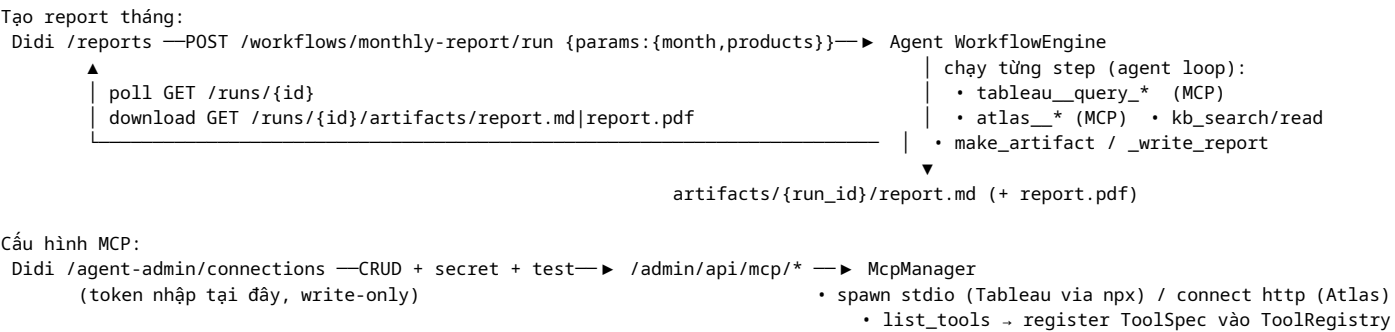
3. Mục tiêu & phạm vi

In-scope - A. Cấu hình MCP connections trong Didi (thêm/sửa/bật-tắt/test, nhập & cập nhật token). Agent trở thành **MCP host**, nạp tool của Tableau MCP + Atlas vào tool registry. - **B.** Upload skill/workflow (file .md đơn, hoặc .zip nhiều file) qua Didi; agent dùng nội dung skill để quyết định *làm gì / làm thế nào / ra file gì*. - **C.** Trang Reports + nút “Tạo report tháng”: chọn skill report + tham số (tháng, sản phẩm) → chạy → tải **PDF + Markdown**; tùy chọn lịch hằng tháng + email.

Out-of-scope (giai đoạn này) - Tự build MCP server cho Atlas nếu Atlas chưa có sẵn MCP (sẽ chốt sau — xem §10). - Viết nội dung nghiệp vụ của report (do skill quyết định — Duy/đội cung cấp skill). - Realtime streaming tiến độ chi tiết trên UI (đã có poll + Telegram progress).

4. Kiến trúc To-be

Nguyên tắc: Agent = **MCP host** (vì LLM cần tool lúc inference). Didi = **config plane + trigger + xuất bản**. Token nhập ở Didi → đẩy xuống agent, **lưu mã hoá tại agent**, chỉ dùng để khởi tạo MCP server, **không bao giờ trả lại / log**.



- Dùng **MCP Python SDK** (pip install mcp). Hỗ trợ 2 transport:
 - **stdio** — cho Tableau MCP (npx -y @tableau/mcp-server), Atlas-nếu-là-stdio.
 - **streamable-http / SSE** — cho MCP server chạy dạng service (Atlas nếu expose HTTP, hoặc Tableau MCP chạy sidecar HTTP).
- API nội bộ (async):
 - connect(server_cfg) -> status : mở session, initialize, list_tools().
 - list_tools(server_id) -> [Tool].
 - call_tool(server_id, tool_name, args) -> content.
 - disconnect(server_id), health(server_id).
 - reload() : đọc bằng mcp_servers (enabled=1), kết nối, **đăng ký tool vào ToolRegistry**.
- **Namespacing tool**: f"{server.prefix}__{tool.name}" (vd tableau__query_datasource, atlas__metric_timeseries).
Tránh trùng tên + biết tool thuộc server nào.
- **Vòng đời**: kết nối ở startup (sau khi build services) + mỗi khi save/toggle config. stdio server giữ subprocess sống; có timeout + retry; lỗi 1 server không được làm chết agent.
- **Giới hạn an toàn**: timeout mỗi call_tool (vd 60s), cắt độ dài kết quả theo TOOL_RESULT_MAX_CHARS, whitelist tool (tuỳ chọn ẩn tool nguy hiểm).

A2. Tiêm tool MCP vào ToolRegistry — sửa app/core/tools/registry.py Thêm cơ chế đăng ký động (giữ nguyên _build() cho tool built-in):

```
def register_external(self, specs: list[ToolSpec]) -> None:
    for spec in specs:
        self._tools[spec.name] = spec          # schemas()/execute() tự lộ ra cho LLM

def unregister_prefix(self, prefix: str) -> None:
    for name in [n for n in self._tools if n.startswith(f"{prefix}__")]:
        del self._tools[name]
```

- McpManager build ToolSpec cho mỗi MCP tool: parameters = JSON schema do server trả về; handler = lambda args, ctx: mcp.call_tool(server_id, tool_name, args).
- **Không cần sửa agent_loop.py** (đã dùng tools.schemas()/tools.execute()); nhánh JSON-mode ở agent_loop.py:243 cũng nhận vì tool nằm trong self._tools).
- Nơi wiring: app/main.py:339 (tools = ToolRegistry(...)) → sau đó khởi tạo McpManager(settings, db, tools) và await mcp.reload() trong lifespan startup.

A3. Lưu cấu hình + secret — migration app/db/migrations/0006_mcp.sql (mới)

```
CREATE TABLE IF NOT EXISTS mcp_servers (
    server_id TEXT PRIMARY KEY,          -- slug: 'tableau', 'atlas'
    name TEXT NOT NULL,
    prefix TEXT NOT NULL,                -- namespace tool
    transport TEXT NOT NULL CHECK(transport IN ('stdio','http')),
    command TEXT,                        -- stdio: 'npx'
    args TEXT NOT NULL DEFAULT '[]',     -- JSON array
    base_url TEXT,                       -- http
    env_public TEXT NOT NULL DEFAULT '{}', -- JSON: SERVER, SITE_NAME, PAT_NAME... (không bí mật)
    enabled INTEGER NOT NULL DEFAULT 0,
    status TEXT NOT NULL DEFAULT 'unknown', -- unknown|connected|error
    last_error TEXT, last_checked_at TEXT,
    tool_count INTEGER NOT NULL DEFAULT 0,
    updated_at TEXT NOT NULL, updated_by TEXT
);

CREATE TABLE IF NOT EXISTS mcp_secrets (
    server_id TEXT NOT NULL, secret_key TEXT NOT NULL, -- tách riêng, mã hoá -- vd 'PAT_VALUE', 'API_TOKEN'
    value_encrypted BLOB NOT NULL, updated_at TEXT NOT NULL,
    PRIMARY KEY (server_id, secret_key)
);
```

- **Mã hoá**: thêm app/services/crypto.py dùng cryptography (Fernet/AES-GCM), key từ env mới MCP_SECRET_KEY (≥32 bytes; production bắt buộc — bổ sung vào validator settings.py). Khi spawn MCP server, manager giải mã và đưa vào env của subprocess (stdio) hoặc header Authorization (http). **Secret không nằm trong env_public, không trả ra API, không ghi log/audit-detail.**
- app/settings.py: thêm mcp_secret_key, mcp_enabled (bật/tắt toàn cục), mcp_call_timeout_seconds.

A4. Admin API MCP — thêm vào app/main.py (mới)

Method · Path	RBAC	Body / Ghi chú
GET /admin/api/mcp/servers	viewer	List, ẩn secret (chỉ trả key tồn tại)
POST /admin/api/mcp/servers	operator	{server_id,name,prefix,transport,command,...
PATCH /admin/api/mcp/servers/{id}	operator	cập nhật cấu hình / enable
DELETE /admin/api/mcp/servers/{id}	operator	gỡ + unregister_prefix
PUT /admin/api/mcp/servers/{id}/secret	operator	{secret_key,value} — write-only , mã hoá, không trả về
POST /admin/api/mcp/servers/{id}/test	operator	connect thử → trả {status, tool_count, tools:[name], error?}
GET /admin/api/mcp/servers/{id}/tools	viewer	list tool đang nạp

→ Pydantic models McpServerCreate/Update/Secret; audit qua services.audit.record (không log value secret).

A5. Didi — UI + BFF + RBAC

1. **BFF RBAC** — src/lib/rbac/agent-admin.ts: thêm rule (vì non-GET không khớp rule sẽ bị forbidden mặc định):

```
{ method: "POST", pattern: "/admin/api/mcp/servers", role: "operator" },
{ method: "PATCH", pattern: "/admin/api/mcp/servers/:id", role: "operator" },
{ method: "DELETE", pattern: "/admin/api/mcp/servers/:id", role: "operator" },
{ method: "PUT", pattern: "/admin/api/mcp/servers/:id/secret", role: "operator" },
{ method: "POST", pattern: "/admin/api/mcp/servers/:id/test", role: "operator" },
```

(GET đã được rule GET /admin/api/** → viewer phủ.) BFF tự inject Bearer + X-Acting-User/Role, không cần sửa proxy.

2. **Trang mới** src/app/agent-admin/connections/page.tsx: bảng server (tên, transport, status, tool_count, enabled), form thêm/sửa, ô nhập **token (write-only, type=password)** gọi PUT .../secret, nút **Test** hiện danh sách tool, toggle enable. Dùng apiFetch('/api/agent-admin/mcp/...') + normalizer mới trong src/lib/api/agent-admin.ts (theo đúng pattern normalizeSkills...).
3. **Nav + i18n** — src/components/layout/sidebar.tsx thêm { href: "/agent-admin/connections", label: t('navConnections'), icon: Plug }; thêm key vào src/lib/store/i18n-store.ts.
4. **Credential Vault (tuỳ chọn)**: có thể bỏ qua vì secret sống ở agent. Nếu muốn quản lý tập trung ở Didi, mở rộng CredentialSource (src/lib/credentials/vault.ts) + nối CHECK trong src/lib/migrations.ts (migration mới 0002, rebuild bảng credentials), rồi Didi đẩy secret xuống agent khi lưu. **Khuyến nghị**: giai đoạn 1 lưu thẳng tại agent cho gọn, vault hoá sau.

A6. Đặc thù từng connector

- **Tableau MCP** (@tableau/mcp-server, Node ≥22, stdio): env SERVER, SITE_NAME, PAT_NAME → env_public; PAT_VALUE → mcp_secrets. Tool tiêu biểu (lấy động lúc runtime): liệt kê datasource/workbook, **query-datasource** (VizQL Data Service), đọc metadata, ảnh view. → Đúng nhu cầu “lấy số liệu”.
- **Atlas (atlas.vng.com.vn)**: **cần chốt** Atlas có sẵn MCP server không (§10). 3 kịch bản: (a) có MCP HTTP → cấu hình transport http + token; (b) chỉ có REST API → viết **MCP wrapper mỏng** hoặc thêm **native tool** atlas_query trong registry.py; (c) chính là nguồn data warehouse mà các data-skill (MMF/FD/FS Hub...) đang giả định → có thể thành 1 MCP/sql tool. Plan để ngỏ adapter, không chặn Khối C (Tableau là đủ để ra report đầu tiên).

KHỐI B — Upload Skill / Workflow

B1. Data layer — đã sẵn sàng, không cần migration source='admin' + content_override đã hoạt động; POST /admin/api/skills|workflows đã insert đúng. load() ưu tiên content_override.

B2. Didi — bổ sung UX upload (file .md)

- src/app/agent-admin/skills/page.tsx & workflows/page.tsx (đã POST content_override): thêm nút **“Upload .md”** → đọc file ở client → tự điền name/description/content_override (parse frontmatter --name/description/schedule--- + heading đầu). Validate skill_id/workflow_id khớp `^[a-z0-9_-]+$` (agent sẽ 400 nếu sai).
- Skill nhiều file (.zip):** đi qua đường KB có sẵn — giải nén vào staging theo cấu trúc .agents/skills/<...>/SKILL.md rồi POST /admin/api/kb/delta (push-to-agent) → reload_from_kb(). Tải dùng knowledge-base/push-to-agent. (Skill 1 file → content_override; skill nhiều file → KB.)

B3. Skill report nên là Workflow (không phải Skill) Vì report cần **chạy nhiều bước + ra artifact file**, đúng vai trò Workflow (engine chạy step → report.md). Skill (SKILL.md) hợp với “kiến thức/hướng dẫn nạp vào prompt”. → Report = **workflow content_override** theo template ở §5-C1.

KHỐI C — Monthly Report

C1. Template workflow report (ví dụ — Duy chỉnh nội dung) Khớp parser (workflows/parser.py): frontmatter → các heading thường vào system (đặc tả output), heading ## Quy trình → steps đánh số.

```
---
name: Monthly Wealth Report
description: Báo cáo tháng Wealth Solution. Dùng khi user bấm Tạo report tháng.
schedule: "0 8 1 * *" # 08:00 ngày 1 hằng tháng (tùy chọn)
---
## Mục tiêu
Tổng hợp KPI tháng {month} cho các sản phẩm {products}.

## Nguồn dữ liệu
- Số liệu: tool tableau__* (datasource theo sản phẩm).
- Bối cảnh/định nghĩa: kb_search/kb_read trong "05. Knowledge" và "02. Context".

## Định dạng đầu ra (Output)
Markdown: (1) Exec summary toàn portfolio; (2) mỗi sản phẩm 1 mục: AUM/NAV, user, MoM %, nhận xét; (3) rủi ro/điểm bất thường.

## Quy trình
1. Xác định kỳ báo cáo {month} và danh sách sản phẩm {products} từ tham số.
2. Với mỗi sản phẩm: gọi tableau__query_datasource lấy KPI tháng này và tháng trước; tính MoM.
3. kb_search/kb_read để lấy định nghĩa metric + ghi chú vận hành liên quan.
4. Tổng hợp thành báo cáo theo mục "Định dạng đầu ra"; gọi make_artifact("report.md", <nội dung>).
```

Engine sẽ chạy step → _write_report() tổng hợp report.md. (Có thể để engine tự tổng hợp, hoặc chủ động make_artifact ở step cuối.)

C2. Trang Reports + trigger (Didi)

- src/app/reports/page.tsx (mới): list workflow “report” (lọc theo convention prefix report- hoặc cò), form chọn **tháng + sản phẩm**, nút **“Tạo report tháng”** → POST /api/agent-admin/workflows/{id}/run body {params:{month, products}} → nhận run_id.
- Poll GET /api/agent-admin/runs/{run_id} đến khi status=succeeded → hiện link tải report.md và report.pdf qua GET /api/agent-admin/runs/{run_id}/artifacts/{filename}.
- Nav: thêm mục **Reports** (cho mọi role đăng nhập, hoặc operator+). i18n key mới.

C3. Xuất PDF (engine chỉ ra .md) — quyết định cần chốt (§10)

Phương án	Cách làm	Ưu / Nhược
(Khuyến nghị) PDF tại agent	Thêm step/_write_report hook: markdown+weasyprint render report.md→report.pdf lưu cùng thư mục artifact	Tái dùng nguyên đường tải artifact hiện có; artifact đồng nhất. Nhược: Docker cần lib hệ thống (cairo/pango).
PDF tại Didi	Route src/app/api/reports/[runId]/export thêm puppeteer (nặng) vào fetch report.md qua BFF → markdown-it → PDF (puppeteer)	Không dùng agent. Nhược: thêm puppeteer (nặng) vào Didi.
HTML + in trình duyệt	Agent make_artifact("report.html", ...) có CSS in; Didi "Lưu PDF" qua print	Nhẹ nhất. Nhược: PDF do người bấm in.

→ Đề xuất **(1)**: report ra report.md + report.pdf như artifact, Didi tải về như nhau, ít plumbing mới nhất.

C4. Lịch định kỳ + email (tuỳ chọn)

- Lịch: set workflows.schedule (cron) — cron_scheduler.py đã chạy; report tự tạo hằng tháng + Telegram send_document.
- Email: dùng nodemailer (Didi) — route Didi đọc artifact rồi gửi mail đính kèm; hoặc agent gửi Telegram là đủ. Quyết định theo nhu cầu.

6. Tổng hợp thay đổi theo file

Agent (queo-solution-agent) | File | Thay đổi | |—|—| requirements.txt | + mcp, cryptography, (PDF) markdown, weasyprint | | app/services/mcp_manager.py | **mới** — MCP host (connect/list/call/reload, register tool) | | app/services/crypto.py | **mới** — mã hoá secret (Fernet) | | app/core/tools/registry.py | + register_external(), unregister_prefix() | | app/db/migrations/0006_mcp.sql | **mới** — mcp_servers, mcp_secrets | | app/main.py | + 7 route /admin/api/mcp/*; wiring McpManager ở lifespan/startup (cạnh main.py:339); (PDF) hook render trong engine._write_report | | app/settings.py | + mcp_secret_key, mcp_enabled, mcp_call_timeout_seconds + validator production | | app/core/workflows/engine.py | (PDF) render report.pdf sau report.md | | .env.example, Dockerfile | + env MCP; + Node 22 (nếu chạy studio Tableau in-image) hoặc khai báo sidecar; + lib PDF |

Didi (Didi AI Tool) | File | Thay đổi | |—|—| src/lib/rbac/agent-admin.ts | + 5 rule /admin/api/mcp/* | | src/lib/api/agent-admin.ts | + interface + normalizeMcpServers/Tools | | src/app/agent-admin/connections/page.tsx | **mới** — UI quản lý MCP + token + test | | src/app/reports/page.tsx | **mới** — nút “Tạo report tháng” + poll + tải | | src/app/agent-admin/skills/page.tsx, workflows/page.tsx | + nút Upload .md (điền content_override) | | src/components/layout/sidebar.tsx | + mục **Connections, Reports** | | src/lib/store/i18n-store.ts | + label mới | | (tuỳ chọn) src/app/api/reports/[runId]/export/route.ts | PDF/email nếu chọn render phía Didi |

7. Bảo mật

- **Token MCP**: mã hoá tại agent (mcp_secrets), nhập write-only ở Didi, **không** trả API, **không** log/audit-detail, **không** đưa vào env_public. Tôn trọng GUARDRAIL của Quéo (từ chối lộ token/secret/connection string).
- **RBAC**: đọc = viewer; thêm/sửa/secret/test/run = operator; (cân nhắc) quản lý secret = superadmin. Mặc định non-GET không có rule → BFF trả forbidden (an toàn).
- **Audit**: mọi mutate ghi audit_log 2 phía (Didi audit() + agent services.audit.record), che giá trị secret.
- **Cô lập subprocess MCP**: timeout, giới hạn output, không truyền secret của server này sang server khác; cân nhắc allow-list tool.

- **Artifact:** đường tải đã chặn path traversal (`runs/{id}/artifacts/{filename}` từ chối `/, \`).

8. Triển khai / DevOps

- **Tableau MCP (studio) cần Node 22.** 2 lựa chọn: (a) thêm Node vào image Python (multi-stage) — đơn giản gọi npx; (b) chạy **Tableau MCP sidecar** expose HTTP, agent kết nối transport http — image Python gọn, cô lập tốt hơn. → Khuyến nghị **(b)** cho production, **(a)** cho dev nhanh.
- **MCP_SECRET_KEY** thêm vào secret store (bắt buộc production). Migration 0006 chạy qua `python -m app.cli migrate`.
- **WeasyPrint** (nếu chọn PDF tại agent) cần `libpango/cairo` trong Dockerfile.
- Không đổi cơ chế BFF/AGENT_BASE_URL hiện có.

9. Phân kỳ & cột mốc

Mốc	Nội dung	Kết quả nghiệm thu
M0 (0.5–1 ngày)	Chốt \$10 (Atlas, host Tableau, PDF), tạo MCP_SECRET_KEY	Quyết định kiến trúc xong
M1 — MCP core (3–5 ngày)	mcp_manager, crypto, migration 0006, register_external, wiring startup; test studio bằng Tableau MCP qua CLI	Agent log “connected tableau, N tools”; gọi được 1 tool Tableau từ chat
M2 — Admin API + UI Connections (3–4 ngày)	7 route /admin/api/mcp/* + RBAC BFF + trang connections + Test	Duy thêm Tableau qua UI, nhập PAT, bấm Test thấy danh sách tool
M3 — Report workflow + trigger (2–3 ngày)	Template workflow monthly-report, trang /reports, nút tạo + poll + tải report.md	Bấm nút → ra report.md có số liệu Tableau thật
M4 — PDF + Upload UX + (tuỳ chọn) lịch/email (2–3 ngày)	Render report.pdf; nút upload .md; cron + email	Tải được PDF; upload skill mới chạy được; (tuỳ chọn) report tự chạy đầu tháng

Ước lượng ~2–3 tuần cho 1 dev fullstack; M1 là rủi ro/then chốt nhất.

10. Câu hỏi mở cần Duy chốt (M0)

1. **Atlas (atlas.vng.com.vn):** có sẵn MCP server không? Nếu không, Atlas expose REST hay là data warehouse? (Quyết định adapter — xem A6.)
2. **Host Tableau MCP:** nhúng Node vào image agent, hay chạy sidecar HTTP? (Khuyến nghị sidecar cho prod.)
3. **PDF:** render tại agent (WeasyPrint) hay tại Didi (puppeteer)? (Khuyến nghị agent.)
4. **Secret:** lưu tại agent (gọn) hay vault hoá ở Didi rồi đẩy xuống (tập trung)? (Khuyến nghị agent ở GD1.)
5. **Email & lịch:** có cần auto hàng tháng + gửi mail, hay chỉ bấm thủ công ở GD1?
6. **Quyền:** ai được cấu hình MCP/secret — operator hay chỉ superadmin?

11. Kế hoạch kiểm thử (verify)

- **Unit (agent):** mcp_manager parse tool schema → ToolSpec; crypto mã hoá/giải mã; namespacing; register/unregister_prefix.

- **Integration:** chạy Tableau MCP thật (PAT sandbox) → POST /mcp/servers/{id}/test trả tool_count>0; agent chat gọi tableau__* ra số.
 - **E2E report:** POST /workflows/monthly-report/run → poll succeeded → tải report.md/report.pdf đúng nội dung.
 - **Bảo mật:** secret không xuất hiện trong GET API/log/audit; user thường (viewer) bị 403 khi mutate; guardrail vẫn từ chối khi user hỏi token.
 - **Regression:** Telegram chat + workflow cũ + KB sync không ảnh hưởng.
-

Phụ lục — tham chiếu code đã đọc: Didi vault.ts, agent-admin/[...path]/route.ts, rbac/agent-admin.ts, api/agent-admin.ts, migrations.ts, sidebar.tsx, docs/BACKEND-FRONTEND-HANDOFF.md, docs/AGENT-LIVE-CONFIG.md. Agent app/core/tools/registry.py, app/core/workflows/{engine,parser}.py, app/services/registry.py, app/main.py, app/web/auth.py, app/settings.py, app/db/migrations/000{1,3,4,5}*.sql.